a pure Python solution, there are currently three tools available: Lettuce (http://pythonhosted.org/lettuce), Freshen (https://github.com/rlisagor/freshen), and Behave (http://pythonhosted.org/behave).

These tools are similar, but at the time of writing, Behave is the most stable, best documented, and most feature-rich of the three. Despite the name, Behave has no relationship with JBehave. There's also a Thucydides plugin that allows you to generate Thucydides reports from the Behave test results.

Let's take a closer look at BDD in Python using Behave.

6.6.1 Installing Behave

Unlike the JVM BDD tools we've looked at so far, Behave is primarily a command-line tool. Installing Behave is straightforward and can be done using Pip, the standard Python package installation tool. Pip is the Python equivalent of Gem for Ruby, NuGet for .NET, or npm for Node.js. You use it like this:

```
$ pip install behave
```

This will install the `behave` command-line tool and make the Behave package available to your Python scripts. When this is done, you should be able to run Behave from the command line:

```
$ behave --version
behave 1.2.3
```

6.6.2 The Behave project structure

Behave projects use a simple directory structure, illustrated in figure 6.8. Feature files go in a directory called features, and the step definitions go in a subdirectory of this directory called steps.

6.6.3 Behave step definitions

Behave uses the Gherkin format for its scenarios. In fact, Behave is essentially a Python port of Cucumber, so the feature files are very similar to those used by Cucumber.

Step definitions are written in Python, using `@given`, `@when`, and `@then` decorators, as shown here:

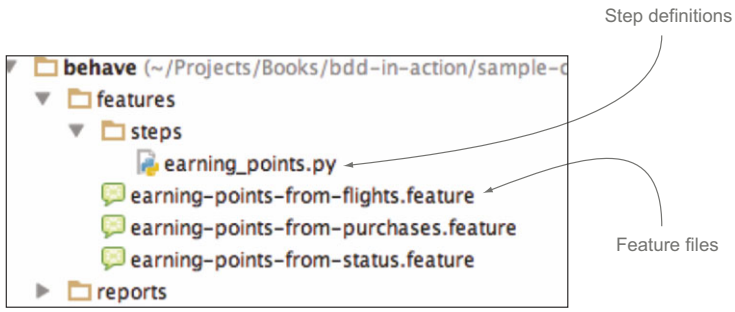


Figure 6.8 The Behave project directory structure